

Introduction to Python Programming

Chapter 15: Classes - Introduction

Introduction

We're about to explore one of the most powerful concepts in programming: **classes**. But before we dive into code, let's start with a thought experiment that will help you understand what classes really are.

1. A Strange Statement: "There Is No Such Thing as a Cow"

Consider this statement: **"There is no such thing as a cow."**

Your first reaction might be: "That's ridiculous! Of course there are cows!" And you'd be right to think that. But I'm not asking you to assume this statement is true. Instead, I want you to explore **how** this statement could be true. What truth is hidden in this seemingly false claim?

The Template vs The Instance

Think about it carefully:

When you say the word "cow," what are you actually referring to?

- Are you thinking of a specific cow standing in a field right now?
- Or are you thinking of the **idea** of a cow - the general concept?

Here's the key insight: When we talk about "a cow," we're usually talking about a **template** - a mental model that exists in our minds. This template includes:

- Four legs
- Produces milk
- Says "moo"
- Eats grass
- Has certain body proportions

This template isn't a real, physical thing. You can't touch it. You can't see it. **The template "cow" exists only as a concept.**

The Specific Instances

Now, **Betsy** - that's a real cow. Betsy is standing in a field somewhere, chewing grass, living her cow life. **Bessie** is another real cow. **Daisy** is yet another.

These are **specific instances** - real, physical cows that exist in the world.

Recognition Through the Template

Here's what's fascinating: Even though cows can be different colors (brown, black and white, spotted), different sizes, different ages - we **recognize them all as cows** because they match the template in our minds.

You can look at a brown cow and a black-and-white cow and instantly know: "Both of those are cows." How? Because both fit the template.

So the truth in "there is no such thing as a cow" is this:

- The general concept "cow" (the template) isn't a physical thing
- What exist are **specific cows** (instances) - Betsy, Bessie, Daisy
- We use the template to create and recognize individual cows

2. Classes: Templates for Objects

This is **exactly** how classes work in programming.

A class is a template. It defines what something is and what it can do, but the class itself isn't a real, usable thing.

An object (or instance) is the real thing. It's created from the template and exists in your program's memory.

The Relationship

```
Class (Template)  →  Objects (Instances)
"Cow"            →  Betsy, Bessie, Daisy
```

Just like:

- The template "cow" allows us to create specific cows
- A **class** allows us to create specific **objects**

3. Creating Your First Class

Let's create a simple `Dog` class:

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

What this means:

- `class Dog:` - This creates the template (the class)
- `__init__` - This is the **initializer** (also called constructor), it runs when you create a new dog
- `self.name` and `self.breed` - These store information about each specific dog

Creating Objects from the Class

```
# Create specific dogs (instances)
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "German Shepherd")
dog3 = Dog("Luna", "Beagle")
```

Now we have three **real** dogs in our program:

- `dog1` is named Buddy
- `dog2` is named Max
- `dog3` is named Luna

Each is created from the same template (`Dog`), but each is unique.

Accessing Object Information

```
print(dog1.name)    # Buddy
print(dog2.name)    # Max
print(dog3.breed)   # Beagle
```

4. Understanding `__init__` (The Initializer)

The `__init__` method is special - it runs automatically when you create a new object.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed
```

Breaking it down:

1. `__init__` - Special method name (double underscores mean "special")
2. `self` - Represents the specific object being created
3. `name, breed` - Information we need to create a dog
4. `self.name = name` - Store the name in this specific object
5. `self.breed = breed` - Store the breed in this specific object

Think of it like this:

- When you call `Dog("Buddy", "Golden Retriever")`
 - Python creates a new dog object
 - Runs `__init__` to set it up
 - Stores "Buddy" as the name and "Golden Retriever" as the breed
 - Returns the completed dog object
-

5. Instance Variables

Variables stored in an object are called **instance variables**.

```
class Dog:
    def __init__(self, name, breed, age):
        self.name = name      # Instance variable
        self.breed = breed    # Instance variable
        self.age = age        # Instance variable
```

Each object has its own copy of these variables:

```
dog1 = Dog("Buddy", "Golden Retriever", 3)
dog2 = Dog("Max", "German Shepherd", 5)

print(dog1.age)  # 3
print(dog2.age)  # 5
```

`dog1` and `dog2` are separate objects with their own data.

6. Multiple Objects from One Template

The power of classes: create many objects from one template.

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

# Create multiple cars
car1 = Car("Toyota", "Camry", 2020)
car2 = Car("Honda", "Civic", 2019)
car3 = Car("Ford", "Mustang", 2021)
```

```
# Each car is unique
print(f"{car1.year} {car1.make} {car1.model}")
print(f"{car2.year} {car2.make} {car2.model}")
print(f"{car3.year} {car3.make} {car3.model}")
```

Output:

```
2020 Toyota Camry
2019 Honda Civic
2021 Ford Mustang
```

Same template (`car`), but three unique objects with different data.

7. Why Classes Matter

Organization

Classes group related data together:

```
# Without classes - messy
dog1_name = "Buddy"
dog1_breed = "Golden Retriever"
dog1_age = 3
dog2_name = "Max"
dog2_breed = "German Shepherd"
dog2_age = 5

# With classes - organized
dog1 = Dog("Buddy", "Golden Retriever", 3)
dog2 = Dog("Max", "German Shepherd", 5)
```

Reusability

Write the template once, create many objects:

```
class Student:
    def __init__(self, name, student_id, grade):
        self.name = name
        self.student_id = student_id
        self.grade = grade

# Create a whole classroom
students = []
students.append(Student("Alice", "001", 85))
students.append(Student("Bob", "002", 92))
students.append(Student("Charlie", "003", 78))
```

Real-World Modeling

Classes let you model real things in code:

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance

checking = BankAccount("123456", 1000.00)
savings = BankAccount("789012", 5000.00)
```

8. Basic Examples

Example 1: Book Class

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

# Create books
book1 = Book("Python Programming", "John Smith", 350)
book2 = Book("Data Science", "Jane Doe", 420)

# Access information
print(f"{book1.title} by {book1.author} - {book1.pages} pages")
```

Example 2: Rectangle Class

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

# Create rectangles
rect1 = Rectangle(10, 5)
rect2 = Rectangle(7, 3)

# Calculate area manually
areal = rect1.width * rect1.height
print(f"Rectangle 1 area: {areal}")
```

Example 3: Person Class

```
class Person:
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name
        self.last_name = last_name
        self.age = age

# Create people
person1 = Person("Alice", "Johnson", 25)
person2 = Person("Bob", "Smith", 30)

# Use the data
print(f"{person1.first_name} {person1.last_name} is {person1.age} years old")
```

9. About `self` (Keep It Simple)

You've noticed `self` appears everywhere in classes. Here's a simple explanation:

`self` represents "this specific object."

When you write:

```
self.name = name
```

You're saying: "Store this name in **this specific object**."

When you create multiple objects:

```
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Max", "German Shepherd")
```

Each time `__init__` runs:

- For `dog1: self` means "dog1"
- For `dog2: self` means "dog2"

`self` is how Python knows which object to store data in.

You don't need to fully understand `self` yet. Just know:

1. Always put `self` as the first parameter in `__init__`
2. Use `self.variable_name` to store data in the object
3. Python handles the rest automatically

10. Practical Example: Student Records

```
class Student:
    def __init__(self, name, student_id, gpa):
        self.name = name
        self.student_id = student_id
        self.gpa = gpa

# Create students
students = []
students.append(Student("Alice", "001", 3.8))
students.append(Student("Bob", "002", 3.5))
students.append(Student("Charlie", "003", 3.9))

# Display all students
print("Student Records:")
print("-" * 40)
for student in students:
    print(f"{student.name} (ID: {student.student_id}) - GPA: {student.gpa}")
```

Output:

```
Student Records:
-----
Alice (ID: 001) - GPA: 3.8
Bob (ID: 002) - GPA: 3.5
Charlie (ID: 003) - GPA: 3.9
```

Key Takeaways

✓ **Classes are templates** - like the concept "cow" or "dog"

✓ **Objects are instances** - like specific cows (Betsy, Bessie) or specific dogs (Buddy, Max)

✓ `class ClassName:` creates the template

✓ `__init__` is the initializer - runs when creating objects

✓ `self` represents the specific object being created

✓ **Instance variables** store data in each object: `self.variable_name`

✓ **Create objects** by calling the class like a function: `obj = ClassName(args)`

✓ **Access data** with dot notation: `object.variable_name`

✓ **One template, many objects** - create unlimited instances from one class

Reflection Questions

1. What does "there is no such thing as a cow" mean in the context of classes?
 2. What's the difference between a class and an object?
 3. What does `__init__` do?
 4. What does `self` represent?
 5. Can you create multiple objects from one class?
 6. How do you access data stored in an object?
 7. What are instance variables?
 8. Why would you use a class instead of just variables?
-

Practice Exercises

1. **Create a Book Class:** Include title, author, and year. Create 3 book objects.
 2. **Create a Car Class:** Include make, model, year, and color. Create 5 different cars.
 3. **Create a Product Class:** Include name, price, and quantity. Create a list of products.
 4. **Create a Circle Class:** Store radius. Create circles with different radii.
 5. **Create a Course Class:** Include course code, name, and credits. Create your current courses.
 6. **Create a Pet Class:** Include name, species, and age. Create your pets (real or imaginary).
 7. **Create a Phone Class:** Include brand, model, and storage. Create different phones.
 8. **Modify Examples:** Take any example from this chapter and add one more instance variable.
-

Looking Ahead

This is just the beginning! Classes become even more powerful when combined with:

- Methods (functions inside classes) to make objects **do** things
- Special methods for more advanced functionality
- Inheritance to create related classes

For now, focus on understanding:

- Classes are templates
- Objects are instances created from templates
- Each object has its own data

Once you're comfortable with this foundation, you'll be ready for more advanced concepts!